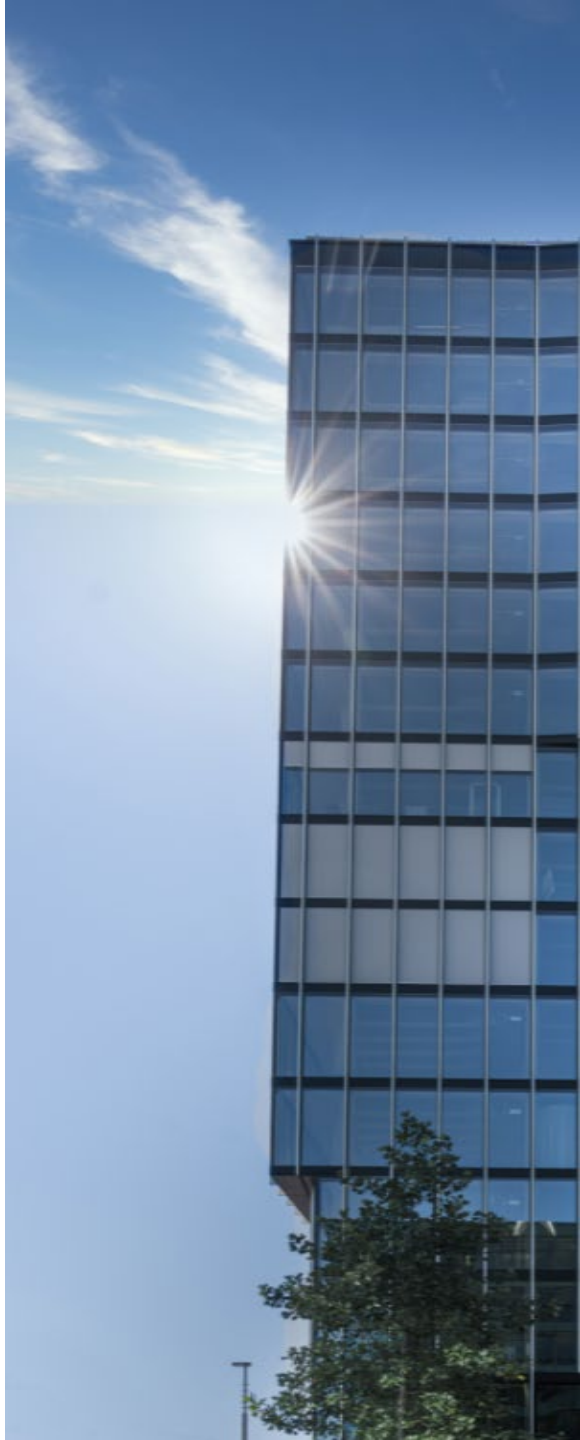
HSLU Hochschule Luzern

Objektorientierte Programmierung

Fehlerbehandlung mit Exceptions

Roland Gisler



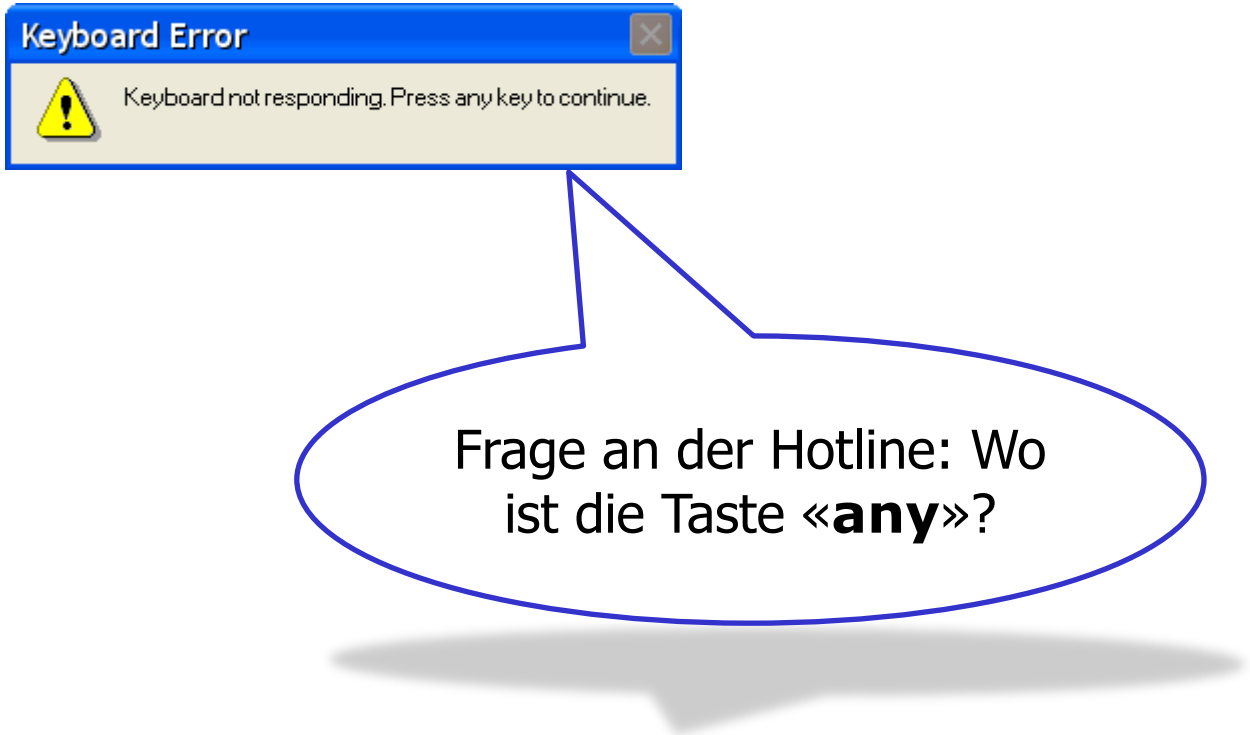
FH Zentralschweiz

Inhalt

- Was ist überhaupt ein Fehler?
- Wie signalisieren wir einen Fehler?
- Herausforderungen des Fehlerhandlings
- Konzept der Ausnahmebehandlung
- Exceptionhandling in Java
- Empfehlungen für gutes Exceptionhandling
- Zusammenfassung

Lernziele

- Sie kennen verschiedene Techniken zur Fehlerbehandlung.
- Sie können verschiedene Fehler(-situationen) abhängig vom Kontext beurteilen.
- Sie kennen das Konzept der Ausnahmebehandlung.
- Sie können in Java Exceptions definieren, behandeln und selber auslösen.


Keyboard Error
Keyboard not responding. Press any key to continue.
Frage an der Hotline: Wo ist die Taste «**any**»?

Was ist überhaupt ein Fehler?

Beispiel einer Fehlersituation

- Eine Software meldet folgenden Fehler:

```
Ein Fehler ist aufgetreten: Die Datei
O11_IP_Exceptionhandling.pptx kann nicht gelesen werden!
```

- Es gibt sehr viele mögliche Ursachen für diesen Fehler:
  - Datei nicht gefunden?
  - Falscher Name verwendet?
  - Ist die Datei leer?
  - Falsches Datenformat?
  - Keine ausreichende Rechte?
  - Dateisystem nicht gemounted?
  - Netzwerk nicht verfügbar?

Identischer Fehler – Aber unterschiedliche Ursachen!

- **Division durch Null** – (`ArithmeticException`)
  - Ist es ein Programmfehler oder ein Eingabedatenfehler?
- **Stack-Überlauf** – (`StackOverflowError`)
  - Ist es ein Fehler im (rekursiven) Algorithmus?
  - Oder haben wir unerwartet (zu) viele Daten?
  - Oder haben wir wirklich (zu) wenig Speicher?
- **Zugriff auf fehlendes Objekt** – (`NullPointerException`)

  Ist es ein Programmierfehler…
  - in der aufrufenden Methode, weil sie das nicht erwartet bzw. nicht prüft?
  - in der aufgerufenen Methode, weil sie das Objekt z.B. nicht erstellen konnte oder nicht gefunden hat?

➔ Fehlerbeurteilung ist **stark abhängig vom** jeweiligen **Kontext**!

Drei grundlegende Fehlerarten bzw. -kategorien

1. **Nicht behebbare** Fehler. ①

   Beispiel: `OutOfMemoryError` (Speichermangel)

   - Weder im Programm noch als Anwender können wir (zur Laufzeit) etwas unternehmen. Einzige Option: Sofortiger Abbruch des Progammes!

2. **D**urch **reine Wiederholung** «behebbar». ②

   Beispiel: `SocketException` (Kommunikationsproblem)

   - z.B. durch einen temporären Netzwerkunterbruch verursacht, wird der Fehler bei einer Wiederholung nicht mehr auftreten.

3. **D**urch **aktive Korrektur** (und dann Wiederholung) behebbar. ③

   Beispiel: `UnknownHostException` (Unbekannter Hostname)

   - Kann z.B. durch die Korrektur/Ergänzung des Hostnamens und nachfolgende Wiederholung behoben werden.

Aspekte der Fehlerbehandlung

- Fehlerbehandlung ist eine **sehr anspruchsvolle** Aufgabe!
  - Wichtig: «***Read the fucking message, but don't shoot the messenger!***»
- Grundsatz: Fehler so **nahe** an der Quelle wie **möglich**, und so weit davon entfernt wie nötig behandeln!
  - Je näher, je mehr Detailinformationen sind noch vorhanden!
  - Je weiter entfernt, je mehr Kontext zur bessern Einordnung ist vorhanden.
- Für jeden einzelne(n) Fehler(-art) sollte explizit betrachtet werden:
  - Wie ist er zu behandeln?
  - Wo soll er behandelt werden? (auf welcher Ebene*)
  - Ist die Delegation an eine höhere Ebene* sinnvoll?
  - Macht eine Nachfrage beim/der Benutzer*in Sinn?
  - Muss die Verarbeitung / das Programm abgebrochen werden?

* im Sinn von aufeinander aufbauenden Softwareteilen/-schichten, jeweils in der Rolle als Nutzende und Anbietende von Methoden.

Wie signalisieren wir Fehler?

Fehlersignalisation über Rückgabewerte

- Wir verwenden den Rückgabewert einer Methode, um zu signalisieren, ob eine Aktion erfolgreich war oder nicht.
- Beispiel:

```
public boolean login(String uid, String pwd) { … }
```

- Der Rückgabewert (**`boolean`**) sagt aus, ob das Login funktioniert hat oder nicht (`true`: Erfolgreich, `false`: nicht erfolgreich).
  - Für (sehr) einfache Szenarien durchaus geeignet.
- Aber was passiert, wenn wir eine Folge von Aufgaben haben, die einzeln misslingen können, aber voneinander abhängig sind? Beispiel:

```
public int doSomethingBig() {
    resultA = doSomethingA();
    resultB = doSomethingB();
    resultC = doSomethingC();
    return …
}
```

Fehlersignalisation über Rückgabewerte - Beispiel

- Was das Fehlerhandling aus dem Beispiel von oben macht:

```
public int doSomethingBig() {
    int result = doSomethingA();
    if (result != 0) {
        handleErrorA(result);
    } else {
        result = doSomethingB();
        if (result != 0) {
            handleErrorB(result);
        } else {
            result = doSomethingC();
            if (result != 0) {
                handleErrorC(result);
            }
        }
    }
    return result;
}
```

Fehlersignalisation über Rückgabewerte - Bilanz

- Hauptproblem: Der **Quellcode** wird **sehr unübersichtlich**.
  - Die Fehlerbehandlung dominiert den Quellcode.
  - Die eigentliche Fachlichkeit wird geradezu verschleiert.
  - Der Rückgabewert wird für den Fehlerstatus «verschwendet».
  - Oder noch «schlimmer»: Sogar mit dem Resultat vermischt (häufig)!
- Bilanz: Die Nachteile überwiegen klar.
  - Speziell bei Sprachen, welche «nur» mit Rückgabewerten Resultate liefern können (Parameter «nur» call by value), ist diese Technik in vielen Situationen unbefriedigend.

  ➔ Aber nicht generell, es gibt auch Ausnahmen!
- Bei manchen Programmiersprachen geht es gar nicht anders!
- Alternative Lösung: Konzept der ➔**Ausnahmebehandlung**.

Konzept der Ausnahmebehandlung

Konzept der Ausnahmebehandlung - Exceptions

- Viele Programmiersprachen bieten explizite Sprachkonstrukte und Schlüsselwörter zur Fehlerbehandlung mit **Exceptions** an.
- Meist gibt es eine Menge vordefinierte Exceptiontypen (Klassen), es können aber auch eigene, spezifische Exceptions definiert werden.
  ➔ Wenn sinnvoll immer bereits vorhandene Typen wiederverwenden!
- Im Fehlerfall werden Exceptions erzeugt (`new`), mit Informationen bestückt und von der Fehlerquelle «geworfen» (`throw`) – die Quelle.
- Exceptions können auf der jeweils **nächsten, höheren Ebene** zur Behandlung «gefangen» (`catch`) werden, oder «weitergereicht» (`throws`) werden oder sogar neu geworfen werden (re`throw`ing) – die Senke/Empfänger.
- Mit gutem Exceptionhandling wird der Code etwas übersichtlicher!
  - Aber: **Exceptions sind auch kein Allheilmittel und nicht perfekt!**

Exceptionhandling – Einfaches Beispiel

- Fehler werden auf logischer Ebene explizit als **Ausnahmen** (Exceptions) definiert und meist zusammengefasst behandelt.

```
public void doSomethingBig() {
    try {
        doSomethingA();
        doSomethingB();
        doSomethingC();
    } catch (SomeException e) {
        handleError(e);
    }
}
```

- Java und andere jüngere Sprachen kennen dieses Konzept.

Exceptionhandling in Java

Java – (Viele) **Schlüsselwörter zum Exceptionhandling**

- `throw`
  - Löst (wirft) eine Exception aus.
- `throws`
  - Reicht eine Exception an die höhere Aufrufebene weiter.

Auslösen

- `try`
  - Markiert einen Block, in welchem Exceptions erwartet werden, die man behandeln (fangen) will (oder muss).
- `catch`
  - Definiert einen Block für die Behandlung eines spezifischen Exceptiontyps (Klasse).
- `finally`
  - Markiert einen optionalen Block, der **immer** ausgeführt wird.

Behandlung

Drei grundlegende Basis-Exceptions in Java

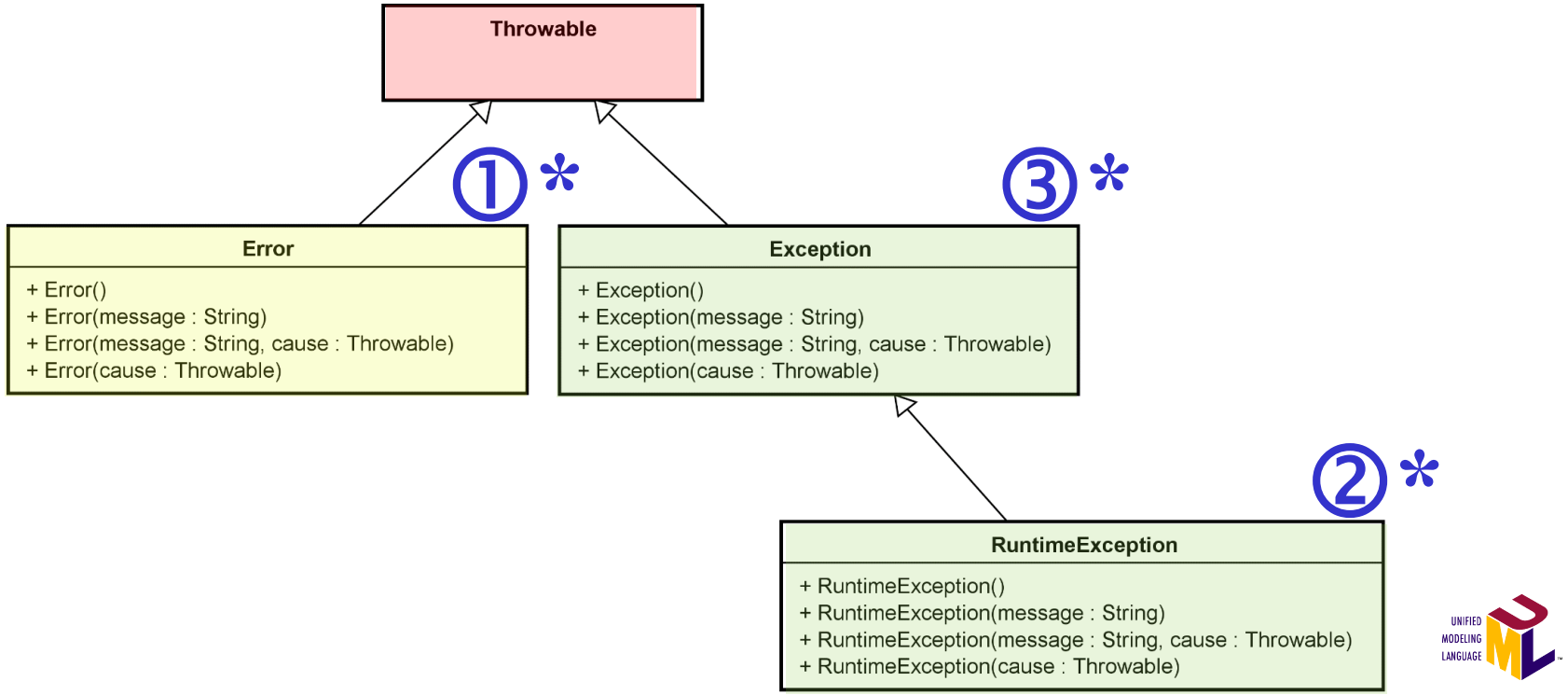
Drei Basisklassen für Exceptions stehen für die drei Semantiken:

- `Error`: Für **nicht behebbare**, schwerwiegende Fehler. ①*
  - in der Regel keine explizite Behandlung (➔**unchecked**).
- `RuntimeException`**:** Für **behebbare**, wenig erwartete Fehler. ②*
  - Behandlung dieser Exceptions ist **optional** (➔**unchecked**).
- `Exception`: Für **behebbare** und (quasi) erwartete Fehler. ③*
  - Behandlung dieser Exceptions wird **erzwungen** (➔**checked**).
- Von allen drei Basisklassen gibt es bereits **eine Menge** von Spezialisierungen für **konkrete** Fehlersituationen!
  - Sinnvolle Wiederverwendung bestehender Typen ist empfohlen.
- **Wichtig**: **Unbehandelte** Fehler führen zur sofortigen **Terminierung** der JVM mit einer Konsolenausgabe (Stacktrace).

* Nummern beziehen sich auf die Fehlerarten auf Slide 8

Basis-Exceptionklassen in Java

- Folgende Basisklassen stehen im Exceptionhandling zur Verfügung: `Error`, `Exception`, `RuntimeException`

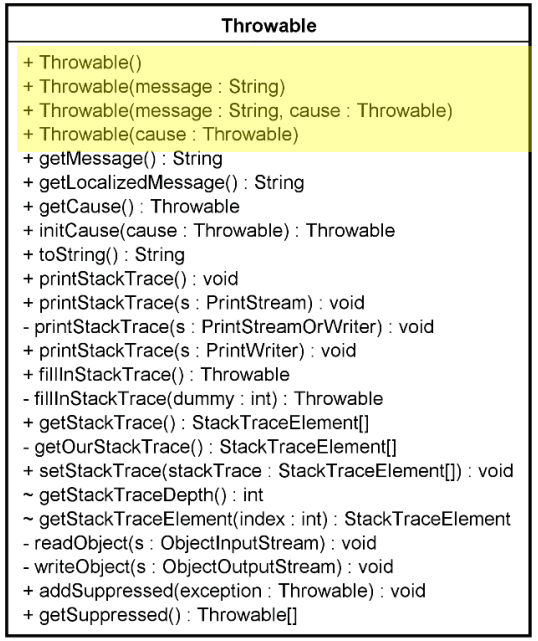


- Eigene Exceptions **spezialisieren** wir typisch von `Exception` oder `RuntimeException`, selten von `Error`. `Throwable` ist **tabu**!

* Nummern beziehen sich auf die Fehlerarten auf Slide 8

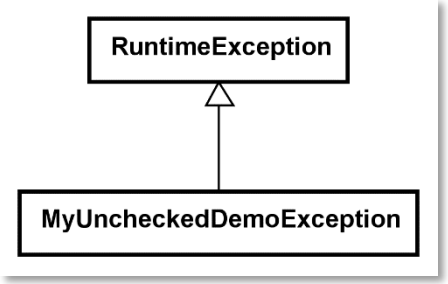
Methoden der Klasse `Throwable` - Auswahl

- **`Throwable`** als Basisklasse aller Exceptions.
- **`getMessage()`**
  Liefert Zusatzinformationen zu einer Exception als **`String`** (für Ausgabe).
- **`getCause()`**
  Liefert die ursprüngliche, auslösende Ausnahme (kann auch eine ganze Kette sein).
- **`printStackTrace(…)` / `getStackTrace(…)`**
  Ermöglichen den Zugriff auf und die Ausgabe des Stacktrace.
- **Wichtig**: Die **gelb** markierten Konstruktoren sollten bei **eigenen** Exceptiontypen in der Regel **überschrieben** werden.



Exceptions auslösen (Quelle)

Werfen von Exceptions: Sofortiger Abbruch der Methode!



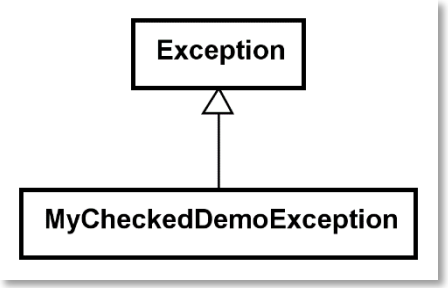
- Beispiel einer **unchecked** (Runtime-)Exception: ②

```
public void throwUncheckedExceptionDemo() {
    LOG.debug("Methode: Anfang");
    …
    if (isSomeFailure()) {
        throw new MyUncheckedDemoException(…);
    }
    LOG.debug("Methode: Ende");
}
```

```
DEBUG - Methode: Anfang
Exception in thread "main" ch.hslu.oop.ExceptionDemo
  at ch.hslu.oop.ExceptionDemo.throwUncheckedException(ExceptionDemo.java:121)
  at ch.hslu.oop.ExceptionDemo.main(ExceptionDemo.java:34)
```

- Wird die Exception geworfen, wird die Methode beim `throw`-Statement **sofort** abgebrochen und verlassen.
  - Der Rest der Methode wird somit **nicht** ausgeführt!
    ➔ Log mit `"Methode: Ende"` wird somit **nicht** mehr ausgeführt.

Werfen von Exceptions: Checked Exception



- Beispiel einer **checked** Exception: ③

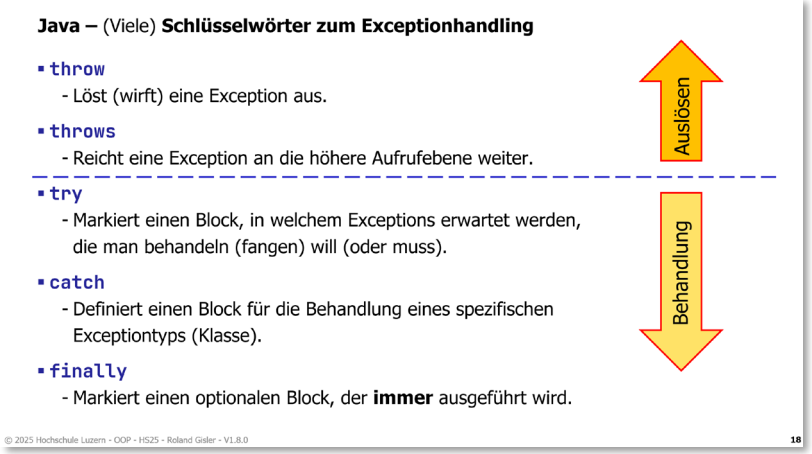
```
public void throwCheckedExceptionDemo()
                        throws MyCheckedDemoException {
    LOG.debug("Methode: Anfang");
    …
    if (isSomeFailure()) {
        throw new MyCheckedDemoException(…);
    }
    LOG.debug("Methode: Ende");
}
```

- **Identisches** Verhalten wie bei einer unchecked Exception: Die Methode wird beim `throw`-Statement sofort abgebrochen.
- **Aber**: Die zusätzliche `throws`-Deklaration im Methodenkopf ist bei **checked** Exceptions **zwingend** erforderlich!

Werfen von Exceptions – Wichtig Hinweise

- Die Entscheidung, **welche** Exception wir werfen, ist immer **abhängig** von der **Fehlerart** und vom **Kontext**.
- Wir werfen eine Exceptions, wenn wir einen unerwarteten, aussergewöhnlichen Fehler **nicht an Ort und Stelle behandeln** können.
  - Weil wir zu wenige Kontextinformationen haben, *oder:*
  - Der Entscheid über die Behandlung delegiert werden soll.
- **Wichtig**: In aller Regel ist der Ort an welchem eine Exception **geworfen** wird (die Quelle) **nie** derselbe Ort (Methode), an welchem die Exception **behandelt** wird!

➔ siehe Slide 18:



Exceptions behandeln (Senke)

Behandlung von Exceptions - Grundlage

- Beachten Sie, dass die Behandlung von Exceptions in aller Regel in einem **übergeordneten**, **aufrufenden** Kontext stattfindet!
  ➔ Unterschiedliche Abstraktionsebenen!

- Die **Behandlung** einer Exception findet darum auch häufig in einer **anderen** Klasse statt als deren **Auslösung**!
  - Übergeordneter Kontext hat die Information und Fähigkeit die Exception korrekt einzuordnen und zu behandeln.

- Beispiel: Auch wenn Sie promovierte Ärzt*in wären: Sollten Sie einen Herzstillstand erleiden, werden Sie sich unmöglich selber den Defibrillator anlegen und auslösen!
  - Ausser natürlich James Bond in «Casino Royale»:
    Der kann natürlich (fast) alles!
    https://www.youtube.com/watch?v=9j9b31PQXKk
    (aber **nicht** während dem Unterricht! ☺)

Eine Exception behandeln

- Beispiel für **die Behandlung** einer (checked) Exception:

```java
public void exceptionCatch() {
    LOG.debug("exceptionCatch: Anfang");
    try {
        LOG.debug(" try-Block: Anfang");
        throwCheckedExceptionDemo("nonumber");  // Wirft Exception!!!
        LOG.debug(" try-Block: Ende");
    } catch (MyCheckedDemoException mcde) {
        LOG.error(" Exception aufgetreten!");
    } finally {                                 // Optionaler Block
        LOG.debug(" finally-Block: Ausgeführt");
    }
    LOG.debug("exceptionCatch: Ende");
}
```

```
DEBUG - exceptionCatch: Anfang
DEBUG -  try-Block: Anfang
ERROR -  Exception aufgetreten!
DEBUG -  finally-Block: Ausgeführt
DEBUG - exceptionCatch: Ende
```

- Funktioniert für checked und unchecked Exception **identisch**.

Wichtige Hinweise zu `try`/`catch`/`finally` - Blöcken

- Es gibt nur **einen** `try`-Block, aber es darf **mehrere** `catch`-Blöcke für **unterschiedliche** Exceptions geben.
- Der **erste** zutreffende `catch`-Block wird als **einziger** ausgeführt!
- **Achtung**: Exceptions sind vererbt und somit **polymorph**! Mehrere `catch`-Blöcke müssen **nach Typ sortiert** werden: Spezialisierte Exceptions **vor** den generalisierten Exceptions, also im hängenden Vererbungsbaum von unten noch oben!
  - Ansonsten «catched» eine generelle die spezifische Exception weg!
- Existiert mindestens ein `catch`-Block, dann ist der `finally`-Block optional: Typisch für "Aufräumarbeiten" vorgesehen.
- Existiert kein `catch`-Block (eher selten) **muss** hingegen ein `finally`-Block folgen.

Exceptions weiterleiten

- **Unchecked** Exceptions (Behandlung ist optional) werden, wenn man sie nicht behandelt, **automatisch** an den aufrufenden Kontext weitergeleitet.
- **Checked** Exceptions (Behandlung ist Pflicht) **muss** man, wenn man sie **nicht** behandeln möchte/kann, **explizit** an den aufrufenden Kontext weiterleiten:
  - Exception muss dafür mit `throws` im Methodenkopf deklariert werden, so als ob sie in dieser Methode geworfen würde!
  - Entscheid wird also bei der Implementation getroffen (statisch).
- In beiden Situationen wird eine Exception bei Nichtbehandlung in letzter Konsequenz von der JVM behandelt:
  - ➔Stacktrace (siehe nächste Folie) auf der Konsole ausgeben.
  - Programm wird mit Fehlerstatus (exit code) **abgebrochen**.

(Unbehandelte) Exceptions - Stacktrace

- Die folgende Ausgabe (Beispiel) bezeichnet man als **Stacktrace**:

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "exit"
        at sun.misc.FloatingDecimal.readJavaFormatString(FloatingDecimal.java:2043)
        at sun.misc.FloatingDecimal.parseDouble(FloatingDecimal.java:110)
        at java.lang.Double.parseDouble(Double.java:538)
        at java.lang.Double.valueOf(Double.java:502)
        at ch.hslu.oop.oop11.ConsoleInput.main(ConsoleInput.java:31)
```

- Die Ausgabe erfolgt auf der Konsole und ist typisch für Exceptions, welche **nicht** behandelt wurden bzw. werden konnten.
- Enthält nützliche und sehr präzise Angaben, wo die Exception aufgetreten ist: **Klasse**, **Methode, Datei** und **Zeilennummer**.
  - Hier ist die Ursache in der `main()`-Methode auf Zeile **31** der Klasse `ch.hslu.oop.oop11.ConsoleInput` zu suchen.
- Tipp: Stacktrace von oben nach unten durchsuchen, die erste «eigene» Klasse (hier `ch.hslu.…`) ist ein guter Einstiegspunkt.

Demo

Empfehlungen für gutes Exceptionhandling

Empfehlungen (1)

- Fehlerbehandlung als **herausfordernde** Aufgabe annehmen!
  - Viele Entwickler*innen gehen viel zu leichtfertig damit um.
- Exceptions in der JavaDoc **unbedingt** dokumentieren!
  - Vor allem die **unchecked** Exceptions!
- Exceptions wirklich «nur» für **echte** Ausnahmen verwenden, und nicht für eigentlich normale, erwartete Spezial- oder Sonderfälle.
- Exceptions sind relativ «teuer» (Ressourcenverbrauch, Laufzeit)
  - Darum **nie** für die «normale» Steuerung des Programmablaufes verwenden (z.B. `ArrayIndexOutOfBound` für Iterationsende).
- Verschachtelung und «rethrowing» von Exceptions zurückhaltend und wohlüberlegt einsetzen.
  - Wenn möglich nur an Schnittstellengrenzen.

Empfehlungen (2)

- Eigene Exceptiontypen: So wenig wie möglich, so viel wie nötig.
  - Wenn immer möglich bereits vorhandene Typen verwenden, deren Semantik somit allgemein bekannt ist.
- **Keine** ineinander verschachtelten `try/catch`-Blöcke!
  - Code wird extrem unübersichtlich und sehr schwer verständlich.
  - Exceptionhandling als eigenständige Aufgabe betrachten (SoC).
- **Niemals leere `catch`-Blöcke implementieren!**
  - Das «Totschweigen» von Ausnahmen ist **extrem** schlecht!

- Fehlerbehandlung immer auch (Unit-)testen (➔Input folgt)!
  - Fehler in der Fehlerbehandlung sind etwas vom Schlimmsten!
- Logging (Input folgt) verwenden, um Fehler festzuhalten.
  - Minimalste Behandlung einer Exception: Loggen (➔Input folgt)!

Zusammenfassung

- Gute Fehlerbehandlung ist keine einfache Aufgabe und erfordert Wissen, Erfahrung und Disziplin.
- Saubere Fehlerbehandlung ist fundamental für eine gute, robuste Software und eine schnelle, zielgerichtete Fehlersuche.
- Konzept der Exceptions: Ausnahmen «werfen», «weiterreichen» oder «fangen» (`throw`, `throws`, `catch`).
- Exceptiontypen in Java: `Error`, `Exception` (checked) und `RuntimeException` (unchecked).

Fragen?